01 / THE MEASURED GAINS

A faster, lighter Quiet News

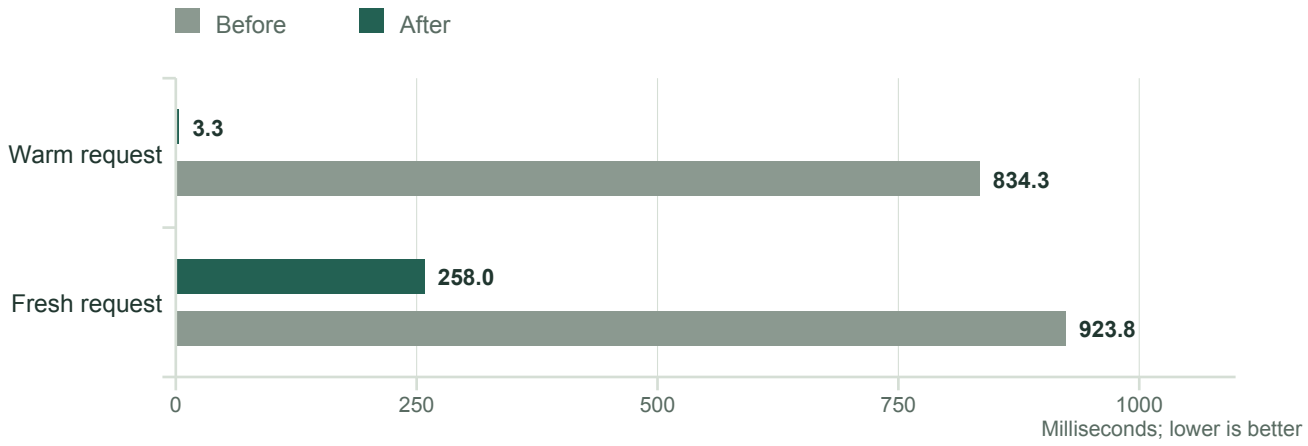
The optimisations cut the work needed to prepare your briefing, reduce data sent over the network and leave more memory available on the Pi. These gains were measured using the same 3,828-article database snapshot.

72%
less time for fresh dashboard processing

97%
smaller history response body

44%
lower peak Node RAM during the test workload

Faster requests, with and without the cache



Medians from requests made on the Pi itself. Fresh requests required a newly prepared snapshot; warm requests could reuse one. These figures measure the news API, rather than the complete page load over your home or mobile connection.

The same briefing, with a more complete archive

All 120 displayed homepage stories were preserved. Removing the history cutoff also made 812 additional grouped stories accessible.

02 / WHERE THE SAVINGS COME FROM

Four changes that remove wasted work

The app retains its existing Node.js, SQLite and browser interface. The changes control when work happens and how much information each request needs.

1 Prepare and reuse the briefing

Headline grouping skips candidates outside its 18-hour matching window. Prepared dashboard snapshots can be reused for up to one minute, with a maximum of 16 entries and a 4 MB estimated memory budget. Database changes invalidate the cache immediately.

Result: fewer repeated calculations and fewer temporary allocations.

2 Filter in the database

Date, publisher, saved and unread filters run in SQLite before article processing. Recent queries use the date index, with an overlap to preserve related coverage. History searches all retained articles; older bookmarks are no longer hidden by a 3,000-row cutoff.

Result: faster selection and a more complete history.

3 Send the information the view needs

History and search responses omit section lists that the browser never displays. Larger news responses use asynchronous gzip compression, with the compressed result reused from the snapshot cache. Titles, summaries and other coverage remain available in the displayed results.

Result: substantially smaller responses, especially in history.

4 Update only what changed

The browser sends an ETag identifying its current briefing. If the response is unchanged, the server returns 304 with no body. When news does change, unchanged sections and cards stay in place. Relative timestamps still advance, and saves or reading-state changes invalidate cached news.

Result: less transfer and redraw work; expanded coverage survives updates.

03 / PRACTICAL IMPACT

What changes in everyday use

Fresh dashboard processing saves roughly two-thirds of a second per measured request. Reusable snapshots make rapid repeat requests much lighter. The benefit you notice also depends on your connection and browser.

ILLUSTRATIVE TRANSFER TIME

Opening history at 1 Mb/s

7.45 s to 0.19 s

Calculated time to transfer one history response body. This excludes network delay, protocol headers, server processing and browser rendering.

ILLUSTRATIVE UNCHANGED POLLING

One visible tab, one quiet hour

5.12 MB avoided

If all 60 one-minute checks are unchanged, 304 responses avoid this much response-body traffic. Headers still travel; fresh stories still trigger updates.

More headroom on a shared Pi

Peak Node memory fell by 102.87 MiB during the workload - about 103 MiB of additional headroom for your other services. Fewer allocations and less request processing should ease contention while you browse.

78.27 s to 0.31 s of Node CPU time

That is 99.6% less CPU time for the 130-request benchmark. The workload deliberately exercises repeated requests and caching. It does not measure the whole Pi's daily CPU use or electricity consumption.

Where the gains should be most noticeable

Rapid repeat visits and multiple tabs, history browsing over a slow Tailscale connection, and browsing while other Pi services are busy. The page also keeps expanded coverage in place, while the archive now returns older relevant stories that were previously excluded.

04 / EVIDENCE AND INTERPRETATION

How the results were checked

Both images ran on the ARM64 Pi with Node 24.22.0, using the same consistent database snapshot in isolated containers. Feed collection was disabled. Measurements used localhost HTTP with gzip support; production bookmarks and preferences were not changed.

Main workload: 40 dashboard requests, 20 history requests, 20 council searches, 10 saved-view requests and 40 unchanged polls, following warm-up requests. A separate test invalidated the cache before each of 10 dashboard requests per image.

MEASUREMENT	BEFORE	AFTER
Fresh dashboard median	923.810 ms	258.049 ms
Warm dashboard median	834.335 ms	3.344 ms
Warm history median	882.843 ms	3.569 ms
History response body	931,433 bytes	24,247 bytes
Dashboard response body	85,352 bytes	29,521 bytes
Unchanged poll body	85,352 bytes	0 bytes (304)
Node CPU time / 130 requests	78.27 s	0.31 s
Peak Node memory (RSS)	233.99 MiB	131.12 MiB

Reliability and limits

The comparison preserved all 120 displayed stories and section deduplication. History grew from 2,918 to 3,730 grouped stories; council search grew from 53 to 62, retaining its previous results. All 18 initial tests passed locally and on the Pi. A later Safari fetch-context fix added a regression test: 19 tests passed locally, three refresh tests passed on the Pi, and the live homepage was verified in Safari.

Warm timings reflect cache hits. The Pi was shared and requests ran sequentially; no concurrent-load or full-browser timing study was performed. RAM was measured from the Node process because Docker reported zero and the Pi kernel did not enforce its configured memory limit. Power use, temperature and storage wear were not measured.

Source: Pi benchmark record, 16 October 2026 | Implementation and Safari fix